

Competitive Programming

Section 10: Greedy II - Invariants And Constructive Problems

Competitive Programming Course

2026-07-04

Table of contents

1	Constructive greedy	3
2	The invariant mindset	4
3	Three patterns	5
4	Worked example 1: repairing an array	6
5	Worked example 2: bracket completion	7
6	Worked example 3: bounded sum sequence	8
7	Detecting impossibility	9
8	Lexicographic constructions	10
9	Proof habit	11
10	C++ implementation notes	12
11	Python implementation notes	13
12	Local judge workflow	14
13	Checklist before submitting	15
14	Exercises	16
14.1	A. Non-Decreasing Repairs	16
14.2	B. Bracket Completion	16
14.3	C. Bounded Sum Sequence	16

1 Constructive greedy

In Section 9, greedy usually selected a subset or paired items. In this section, greedy builds an object:

- repair an array;
- complete a bracket string;
- construct the lexicographically smallest sequence with a required sum.

The contest statement often says:

```
Construct any object satisfying the rules.
```

or:

```
Construct the lexicographically smallest object satisfying the rules.
```

The main danger is making a locally nice choice that traps the suffix. The main tool is a **feasibility invariant**.

2 The invariant mindset

An invariant is a sentence that remains true after every step.

For constructive greedy, the useful invariant is often:

`The prefix I have built can still be extended into a full valid answer.`

When you also need lexicographically smallest output, the rule becomes:

`At this position, choose the smallest value that keeps the suffix feasible.`

That sentence is powerful. It turns a construction problem into repeated local feasibility checks.

3 Three patterns

Pattern	Invariant	Local choice	Exercise
Repair prefix	The processed prefix is valid and minimal	Raise only what must be raised	Non-Decreasing Repairs
Quota plus validation	Exact counts are met and prefixes stay legal	Use earliest smaller characters, then validate	Bracket Completion
Feasible suffix interval	Remaining sum lies in possible suffix range	Choose smallest value preserving range	Bounded Sum Sequence

4 Worked example 1: repairing an array

Array:

```
3 2 5 1 7
```

We may only increase elements. We want a non-decreasing array with minimum total increase.

Scan left to right:

index	value	required minimum	action	cost
1	3	3	keep 3	0
2	2	3	raise to 3	1
3	5	3	keep 5	1
4	1	5	raise to 5	5
5	7	5	keep 7	5

Final repaired array:

```
3 3 5 5 7
```

Why not raise earlier elements more? Because only future elements must be at least the previous repaired value. Making a prefix larger than necessary can only make the suffix harder.

Invariant:

After processing i elements, the prefix is non-decreasing and has the minimum possible last value among all minimum-cost repairs of that prefix.

5 Worked example 2: bracket completion

We want a completed bracket string that is balanced at the end and has positive balance at every strict prefix.

String:

```
??()
```

Length is 4, so a valid final string needs two '(' and two ')'.
Current counts:

```
open:  1
close: 1
need:  one '(' and one ')'
```

Because '(' < ')', the lexicographically smallest fill uses '(' as early as possible:

```
(( ))
```

Balance trace:

```
char:   (  (  )  )
balance: 1  2  1  0
```

Every strict prefix has positive balance, and the final balance is zero.

Now compare:

```
()??
```

The only quota-respecting lexicographic fill is:

```
(( ))
```

Its balance becomes zero after the second character, which is a strict prefix. So it is not a single valid block for this exercise.

6 Worked example 3: bounded sum sequence

Construct the lexicographically smallest length $n = 3$ sequence with values in $[1, 5]$ and sum 9.

At each position, choose the smallest value that still lets the rest reach the remaining sum.

```
position 1:
remaining sum = 9
two positions left, each at most 5
smallest x with  $9 - x \leq 10$  is  $x = 1$ 

position 2:
remaining sum = 8
one position left, at most 5
smallest x with  $8 - x \leq 5$  is  $x = 3$ 

position 3:
remaining sum = 5
no positions left after this
 $x = 5$ 
```

Answer:

```
1 3 5
```

The feasibility interval for a suffix of length k is:

```
 $k \leq \text{suffix\_sum} \leq k * m$ 
```

7 Detecting impossibility

Constructive greedy often starts with a global feasibility check.

For bounded sums:

```
n <= s <= n*m
```

If this fails, no sequence exists.

For bracket completion:

```
n is even  
number of '(' <= n/2  
number of ')' <= n/2
```

Even if quotas are possible, prefix validation may still fail.

For non-decreasing repairs, impossibility never happens because increments are unlimited.

8 Lexicographic constructions

To construct the lexicographically smallest answer, process from left to right.

At position i :

```
try candidates from smallest to largest
choose the first candidate that keeps completion possible
```

Do not ask whether the candidate is locally pretty. Ask whether the suffix still has enough capacity.

For bounded sums, the direct formula is:

```
left = positions after i
x = max(1, remaining_sum - left * m)
```

This is the smallest value that does not leave too much sum for the suffix.

9 Proof habit

Constructive greedy proofs usually use one of these:

1. **Prefix minimality.** The greedy prefix is no larger or no more costly than any other feasible prefix.
2. **Suffix feasibility.** The greedy choice preserves the exact range of possible completions.
3. **Quota exhaustion.** A character or value must be used a fixed number of times, so using the smaller option early is lexicographically optimal if validation still succeeds.

The proof should match the code. If the code chooses a smallest value because of a suffix range, the proof must mention that range.

10 C++ implementation notes

Use `long long` for accumulated costs and sums.

For constructive output, build a vector or string and print it once:

```
vector<long long> ans;  
ans.reserve(n);
```

For bracket strings, validation is a single balance scan:

```
int balance = 0;  
for (int i = 0; i < n; ++i) {  
    balance += (s[i] == '(' ? 1 : -1);  
    if (balance < 0) fail;  
    if (i + 1 < n && balance == 0) fail;  
}
```

11 Python implementation notes

For large output, append strings to a list and join at the end.

For bounded sequences:

```
ans.append(x)
remaining -= x
```

For bracket completion, count quotas first:

```
need_open = n // 2 - s.count("(")
need_close = n // 2 - s.count(")")
```

Then fill question marks from left to right.

12 Local judge workflow

Run Section 10:

```
python3 sections/10_greedy_constructive/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/10_greedy_constructive/check.py
```

The random tests for constructive-output problems brute force small instances and compare against the required canonical output. That is why these local tasks ask for lexicographically smallest answers rather than arbitrary valid answers.

13 Checklist before submitting

- Write the invariant in one sentence.
- Check global feasibility before construction.
- For lexicographic output, prove each chosen value is the smallest extendable value.
- Validate the final object, especially for bracket and prefix constraints.
- Use wide integers for sums and operation counts.
- Stress test small cases with brute force when there are many possible outputs.

14 Exercises

14.1 A. Non-Decreasing Repairs

Use prefix minimality: repair only the current violation.

14.2 B. Bracket Completion

Use quotas plus a prefix-balance invariant.

14.3 C. Bounded Sum Sequence

Use a suffix feasibility interval to construct the lexicographically smallest sequence.